

Lesson 4 — When Things Touch

One lesson: 2 hours 30 minutes · keep this sheet

Name

Class

Date

Cheat sheet

The AABB test

```
function boxesOverlap(a, b) {
  return (
    a.x < b.x + b.w &&
    a.x + a.w > b.x &&
    a.y < b.y + b.h &&
    a.y + a.h > b.y
  );
}
```

All four, joined by `and`.

How to remember it

Do not ask when they *do* overlap. Ask **when they definitely cannot**: a is entirely left of b, entirely right, entirely above, or entirely below. Those are the only four escape routes. Flip them and you have the test.

Boxing a circle

```
function ballBox(ball) {
  return {
    x: ball.x - ball.radius,
    y: ball.y - ball.radius,
    w: ball.radius * 2,
    h: ball.radius * 2
  };
}
```

Drawn as a circle, collides as a square. That is a **hitbox**.

Circles

```
const dx = b.x - a.x;
const dy = b.y - a.y;
const d = Math.sqrt(dx*dx + dy*dy);
return d < a.radius + b.radius;
```

distance = $\sqrt{dx^2 + dy^2}$ — Pythagoras. Skip the square root when only comparing: `dx*dx + dy*dy < limit*limit`

Responding

```
if (boxesOverlap(ballBox(ball), paddle)) {
  ball.y = paddle.y - ball.radius; // FIRST
  ball.speedY = -ball.speedY;      // THEN
}
```

Position before velocity, or it sticks and vibrates.

Steering the ball (design, not physics)

```
const centre = paddle.y + paddle.h / 2;
const off = (ball.y - centre) / (paddle.h / 2);
ball.speedY = off * 320;
```

`off` is `-1` at the top, `0` in the middle, `+1` at the bottom. This is what turns Pong from standing in the right place into **aiming**.

Words

AABB — axis-aligned bounding box. **Hitbox** — the shape you test, not the shape you see. **Tunnelling** — passing through because one step was too big.

Section A — Recall

A1. What does AABB stand for, and what does “axis-aligned” rule out?

[2 marks]

A2. Write out the four conditions for two boxes overlapping.

[3 marks]

A3. Explain the “ask the opposite question” trick for remembering the test.

[3 marks]

A4. Two circles have radii 20 and 30. Their centres are 45 pixels apart. Are they touching? Show how you know.

[3 marks]

A5. Give one reason a game might deliberately make the player's hitbox *smaller* than the picture of the player.

[2 marks]

Section B — Predict the output

B1. Box `a` is at `{x:10, y:10, w:50, h:50}`. Box `b` is at `{x:55, y:70, w:40, h:40}`. Work out each of the four conditions separately, then say whether they overlap.

[4 marks]

B2. A ball at `(100, 100)` has radius 12. What are the `x`, `y`, `w` and `h` of its hitbox?

[3 marks]

B3. Someone changes one `&&` to `||` in `boxesOverlap`. Describe what the player would see happen to the ball.

[4 marks]

B4. A ball travels at 2400 pixels per second. The game runs at 60 fps. The wall is 14 pixels thick. How far does the ball move in one frame, and what will happen when it reaches the wall? Name the problem. [4 marks]

Section C — Find and fix the bug

C1. The ball reaches the paddle and then shivers inside it instead of bouncing away. What is missing? You have seen this bug before — where? [4 marks]

```
if (boxesOverlap(ballBox(), paddle)) {  
    ball.speedX = -ball.speedX;  
}
```

C2. The ball bounces off the paddle correctly most of the time, but occasionally gets caught and bounces twice in a row, ending up behind the paddle. The position fix is present. What other condition is needed, and why? [4 marks]

C3. This collision test never returns true, no matter where the ball is. Why? [3 marks]

```
function ballBox(ball) {  
    return { x: ball.x, y: ball.y, w: ball.radius, h: ball.radius };  
}
```

Hint: there are *two* mistakes here, and only one of them is about the size.

Section D — Build it

Start from your lesson 3 paddle, with the lesson 2 ball bouncing around.

- ☐ **Checkpoint 1.** Write `boxesOverlap(a, b)` and test it by logging the result to the console. Move the ball past the paddle and watch the value flip.
- ☐ **Checkpoint 2.** Write `ballBox()` so the circular ball has a rectangular hitbox. Add a debug view: hold a key to draw every hitbox as an outline.
- ☐ **Checkpoint 3.** Make the ball bounce off the paddle. Fix the **position first**, then flip the velocity.
- ☐ **Checkpoint 4.** Add a second paddle on the far side. Make it follow the ball automatically — but *more slowly than the ball can travel*, or it can never be beaten.

- ☐ **Checkpoint 5.** Add the steering: where the ball hits the paddle should change the angle it leaves at. Play a rally with `STEER_STRENGTH` set to 0, then to 320, and write down which is more fun.
- ☐ **Checkpoint 6.** Make the ball speed up slightly on every paddle hit. Find a rate that makes rallies exciting rather than impossible.

THE DEBUG VIEW IS NOT OPTIONAL

Checkpoint 2 looks like extra work and is not. Being able to see the hitboxes turns every

collision bug for the rest of this course from a mystery into something you can simply look at. Real developers all build one.

Section E — Design challenge

NO RIGHT ANSWER

Marked on reasoning. Every fix for these problems costs something — name the cost.

E1. Tunnelling. Your collision test correctly answers "are these overlapping *right now*?" But the game only looks 60 times a second. Sketch the positions of a fast ball on three consecutive frames to show how it can get from in front of a wall to behind it without ever being inside it.

Three frames of a fast ball approaching a thin wall

E2. Invent a fix. You may change anything.

E3. Now critique your own fix. What does it cost — in speed, in complexity, or in how the game feels?

E4. Too many tests. Imagine 500 bullets and 200 enemies. That is 100,000 collision tests every frame — six million a second. Without looking anything up, invent a way to avoid most of them. You may change how the objects are stored.

Hint: if you knew a bullet was in the top-left corner of the screen, which enemies could you skip without testing at all?

Stretch goals

1. Add three bricks that disappear when hit. You now have the core of Breakout.
2. Write a proper circle-against-rectangle test instead of boxing the ball. (Find the point on the rectangle closest to the circle's centre, then measure to it. Harder than it sounds.)
3. Set the ball speed to 3000, watch it tunnel, then fix it. Any fix that works counts.
4. Give the paddle a hitbox 20 pixels taller than it is drawn. Play for a minute. Does the game feel better or worse? Can you tell it is happening?